

Backend Engineer Career Learning Checklist & Revision Tracker

A structured roadmap for learning, practicing, revising, and growing toward senior backend / system design engineering roles.

Main strategy: Do not learn everything at once. Build mastery in layers: foundations → backend engineering → system design → DevOps → AI-enabled backend engineering. Every learned topic should be revised and practiced using spaced repetition.

How to Use This Document

- Select only 2–3 active learning topics per week.
- For every topic, record the first learning date, revision dates, and practice status.
- Use the checklist to track learning progress and the revision table to avoid forgetting.
- Apply topics in portfolio projects whenever possible.

Learning Depth Guideline

Area	Recommended Depth	Reason
Data Structures & Algorithms	Deep	Build problem-solving power and interview readiness.
System Design	Deep	Essential for senior backend and high-level engineering roles.
DBMS	Deep	Backend systems depend heavily on database design and performance.
Backend Engineering	Deep	Your main professional track.
OS & Networking	Medium	Needed for debugging, performance, and system design understanding.
DevOps, Docker, CI/CD	Medium	Required for modern backend deployment and production workflows.
Kubernetes	Basic to Medium	Know concepts first; go deeper when your job/project needs it.
AI/ML/DS	Basic to Medium	Focus on AI integration and backend use cases, not full research depth initially.

Phase 1 — Core Computer Science

1. Data Structures

- ☐ Arrays
- ☐ Linked List
- ☐ Stack
- ☐ Queue
- ☐ Hash Table
- ☐ Tree
- ☐ Binary Search Tree
- ☐ Heap / Priority Queue
- ☐ Graph
- ☐ Trie
- ☐ Segment Tree
- ☐ Disjoint Set Union

Target: Implement from scratch in C++, understand time/space complexity, solve practical problems, and know real-world use cases.

2. Algorithms

- ☐ Sorting: Quick Sort, Merge Sort, Heap Sort
- ☐ Searching: Binary Search
- ☐ Graph: BFS, DFS, Dijkstra, Topological Sort, Minimum Spanning Tree
- ☐ Dynamic Programming: Knapsack, LIS, LCS
- ☐ Greedy Algorithms
- ☐ Backtracking

Target: Solve problems consistently, explain why the algorithm works, and analyze complexity without memorizing blindly.

3. Complexity Analysis

- ☐ Big O, Big Omega, Big Theta
- ☐ Space complexity
- ☐ Amortized complexity
- ☐ Complexity of hash maps, trees, heaps, recursion, and dynamic arrays

Phase 2 — Computer Science Fundamentals

4. Operating System

- ☐ Processes vs Threads
- ☐ Concurrency
- ☐ Locks and Mutex
- ☐ Deadlock
- ☐ Scheduling
- ☐ Memory Management
- ☐ Virtual Memory and Paging
- ☐ Linux process basics and context switching

5. DBMS

- ☐ Normalization
- ☐ Transactions
- ☐ ACID
- ☐ Indexes
- ☐ Query Optimization
- ☐ Joins
- ☐ Locks
- ☐ Isolation Levels
- ☐ Sharding
- ☐ Replication
- ☐ CAP Theorem

6. Networking

- ☐ OSI Model

- ☐ TCP vs UDP
- ☐ DNS
- ☐ HTTP/HTTPS
- ☐ TLS
- ☐ Load Balancing
- ☐ CDN
- ☐ Reverse Proxy
- ☐ Browser to server request lifecycle

Phase 3 — Software Engineering Foundations

7. OOP and SOLID

- ☐ Encapsulation
- ☐ Inheritance
- ☐ Polymorphism
- ☐ Abstraction
- ☐ Single Responsibility Principle
- ☐ Open/Closed Principle
- ☐ Liskov Substitution Principle
- ☐ Interface Segregation Principle
- ☐ Dependency Inversion Principle

8. Design Patterns and LLD

- ☐ Singleton
- ☐ Factory
- ☐ Builder
- ☐ Adapter
- ☐ Decorator
- ☐ Strategy
- ☐ Observer
- ☐ Command
- ☐ Apply patterns in C++ and real backend examples

9. UML

- ☐ Class Diagram
- ☐ Sequence Diagram
- ☐ Activity Diagram
- ☐ Component Diagram

Phase 4 — Backend Engineering

10. API Design

- ☐ REST API Design
- ☐ Pagination

- ☐ Filtering and Sorting
- ☐ API Versioning
- ☐ Rate Limiting
- ☐ GraphQL Basics

11. Authentication and Security

- ☐ JWT
- ☐ OAuth2
- ☐ Session Auth
- ☐ SSO Basics
- ☐ SQL Injection
- ☐ XSS
- ☐ CSRF
- ☐ API authentication attacks and prevention

12. Caching and Message Queues

- ☐ Redis
- ☐ Memcached
- ☐ Cache Aside
- ☐ Write Through
- ☐ Write Back
- ☐ RabbitMQ
- ☐ Kafka Basics
- ☐ Pub/Sub
- ☐ Event Driven Architecture

Phase 5 — System Design

13. High Level Design

- ☐ Load Balancer
- ☐ CDN
- ☐ Cache Layer
- ☐ Database Scaling
- ☐ Queue System
- ☐ File Storage
- ☐ Observability and Monitoring

14. Distributed Systems

- ☐ CAP Theorem
- ☐ Consistency Models
- ☐ Eventual Consistency
- ☐ Leader Election
- ☐ Distributed Locking
- ☐ Idempotency

- ☐ Retry and Timeout Strategies

15. System Design Practice Problems

- ☐ URL Shortener
- ☐ YouTube / Video Platform
- ☐ Uber / Ride Sharing
- ☐ News Feed
- ☐ Chat System
- ☐ Payment System
- ☐ Notification System
- ☐ Study Tracker / Revision System

Phase 6 — DevOps and Production Readiness

- ☐ Git and GitHub Workflow
- ☐ GitHub Actions
- ☐ CI/CD Pipeline
- ☐ Docker
- ☐ Docker Compose
- ☐ Nginx
- ☐ Linux Server Basics
- ☐ AWS EC2 Deployment
- ☐ Kubernetes: Pods, Services, Deployment, Ingress
- ☐ Logging and Monitoring

Phase 7 — Future-Proof Backend Skills

16. Python, AI Engineering, and ML Basics

- ☐ Python Basics
- ☐ Python for Automation
- ☐ API integration with Python
- ☐ LLM APIs
- ☐ Embeddings
- ☐ Vector Databases
- ☐ RAG Architecture
- ☐ AI-based Backend Engineering
- ☐ ML Basics: regression, classification, neural networks

Weekly Study Strategy

Rule: Keep only 2–3 active topics per week. Example combination:

Day	Focus	Example Task
Monday	DSA	Solve 2–3 problems and revise one data structure.
Tuesday	System Design	Study one design concept and draw a diagram.
Wednesday	Backend Project	Apply API, DB, cache, or authentication concept.
Thursday	DSA Practice	Solve problems from the same pattern.
Friday	Revision	Revise previous topics using notes and examples.
Saturday	Project Build	Implement one production-like feature.
Sunday	Review & Documentation	Summarize learning and update tracker.

Spaced Repetition Schedule

Step	When	Action
First Learn	Day 0	Understand concept, write notes, and implement/example it.
Revision 1	Day 1	Quick review and solve/implement one small example.
Revision 2	Day 7	Review from memory, then check notes.
Revision 3	Day 30	Practice deeper problem or project application.
Revision 4	Day 90	Final long-term retention check and documentation.

Manual Learning & Revision Tracker Template

[illegible]

Priority Ranking for Your Career

Priority	Focus Area	Why It Matters
1	Backend Engineering	Your current professional base and strongest career direction.
2	System Design	Required for senior roles, architecture discussions, and remote interviews.
3	DBMS + Performance	Critical for scalable backend systems.
4	DSA + Problem Solving	Improves thinking and helps in interviews.
5	DevOps + CI/CD	Shows production readiness and improves portfolio strength.
6	AI Backend Engineering	Future-proof skill for AI-powered products.

Golden Rule

Learn → **Practice** → **Build** → **Teach**. For every important topic, learn the concept, practice it, use it in a project, and explain it in your own words. This cycle creates long-term mastery.